

Samana QMS: final production architecture

22 September 2026 · Third and final report of the day · Code at commit 9295489 · Written for management, engineering, and whoever reviews this next

The system now has the architecture the external review asked for, built properly rather than bolted on. The API is its own service, unreachable from the internet. The web tier serves screens and holds no credential of any kind. Prisma is the master description of the data and the typed client for every record, and the queue engine's rules stay in PostgreSQL behind typed, validated calls. Every one of those statements is enforced by a test, not by a convention.

Two of the review's prescriptions were declined a second time, on evidence: rewriting the queue rules in application code, and using Prisma to run migrations. Section 5 gives the reasons and what each would cost. The rest of the review's document is now satisfied line by line, and the one part of §03 that was a genuine code fault, results asserted rather than checked, is closed.

3 services: web tier, private API, scheduler	0 credentials on the internet-facing tier	203 automated tests across 19 files, two runtimes
13 tables described by Prisma, pinned by a drift test	1 real defect the new data tests caught before production	85 overall score, from 82 this morning and 69 on 21 September

1. The architecture

Tier	Process	Holds	Talks to
Browser	Staff screens, TVs, customers' phones	A first-party session cookie	One origin only: the web tier. It never learns the API's address.
Web tier	Next.js standalone server (<code>dist-node</code>)	Its origin and the API's private address. No database, Salesforce or signing credential.	Serves pages with a per-request nonce policy; forwards every <code>/api</code> request to the API over the private network, adding the client address its own edge appended as <code>x-client-address</code> .
API service	Plain Node HTTP server hosting the declared route table (<code>dist-api</code>)	The database and Salesforce credentials, the signing secrets, the Prisma client, the connection pool, the change listener	PostgreSQL and Salesforce. Not reachable from outside: a Render private service, an unpublished container in Compose.
Scheduler	Node worker (<code>dist-worker</code>)	The same database and integration settings, copied from the API	PostgreSQL directly, on two timers: routing every 15 seconds, deliveries and retention on their own cadence. Salesforce and SMS for deliveries.
Database	PostgreSQL 14 or newer, any server	Tables described by <code>prisma/schema.prisma</code> ; the queue engine's functions, triggers and partial indexes, described by <code>db/schema.sql</code>	Both descriptions are generated from the migrated database and each has a test that fails when it goes stale.

One request, end to end: the browser posts to `/api/tickets` on the web origin with its cookie; the proxy refuses hidden paths, sets `x-client-address` and rewrites the request to the API's private address; the API checks the origin, the session and the role from the route table's declaration, validates the body, calls `qms.issue_ticket` through a typed wrapper, validates the returned row against its schema, and answers with a request id; the database raises the change notification that the API relays to every open screen.

2. The reviewer's document, read against this code

"Literal" is how much of what the reviewer wrote is now true of the code, including their prescriptions. "Outcome" is how much of what they were trying to achieve is achieved. Weighted by their own severity labels. The morning's figures were 73 and 84.

§	Reviewer's headline	Literal	Outcome	What stands now
02	There is no backend	95	98	A separate, private API service hosts the route table; the web tier forwards to it and holds nothing. Only the reviewer's wish for a separate deploy schedule per tier is met partially: the tiers deploy as separate services from one repository.
03	No Prisma, no database layer	80	88	Prisma is the master description and the typed client for records; every raw result is validated, so "trust me, it's a T" is gone. Rules stay in PostgreSQL by decision (section 5).

§	Reviewer's headline	Literal	Outcome	What stands now
04	Migrations compound it	80	90	Checksums, one current definition, tested parser, and now a second generated description of the data. Forward-only by policy; Prisma Migrate declined (section 5).
05	Not built on Next.js	100	100	Next.js 16.3.5, and now used only for what it is good at: the screens.
06	Neon serverless	90	95	Standard driver, one pool per process, every connection pinned to UTC; runs on the company server today.
07	One queue, many people	80	90	Push, plain errors, 45-second presence. Lock kept and measured.
08	Roles from nowhere	70	75	One definition shared by server and browser; validated ids; audited sign-in. No SSO or MFA: an IT decision.
09	No consistent standard	88	88	Server, web and data layers each in their own place with a boundary test. The shell's size and three stylesheets remain.
10	How a customer is given to staff	20	30	Business decisions, unchanged: five routing questions with recommendations await the managers.
11	Screen by screen	90	92	29 of 33 fixed, 2 not defects, 2 need devices.
12	Idle load	90	95	2,300 queries a minute at rest is about 330.
13	Salesforce and audit	95	95	Walk-in fallback, breaker, reuse, sign-in and presence audited, history exported.
	Weighted overall	80	85	

Architecture scores, out of 100, before and after this round

Dimension	Morning	Now	What moved it
Back end	86	90	Its own service; typed data layer; validated raw results; one pool for both access styles.
Database	90	92	Two generated descriptions pinned by tests; UTC on every connection.
Security	86	90	No credential on the internet-facing tier; the API has no public address; client address cannot be forged.
API design	84	86	Wire behaviour proven over real HTTP: JSON on every error, request ids, size and content-type limits.
Testing	82	86	Server, data-layer and drift suites; a real defect caught.
Operations	84	86	Three images, three services, the API runs the migrations; bounded graceful shutdown.
Maintainability	80	84	Handlers read as intent; SQL lives in one folder.
Front end	76	76	Unchanged this round.
Observability	68	70	Per-service start and shutdown events; still no metrics or tracing.
Overall	82	85	

3. What changed in this round

Change	What it is	Enforced by
API service New	<code>server/api.ts</code> hosts the unchanged route table on a plain Node HTTP server, validates its configuration before it listens, and drains its connections on stop. Built to one file by <code>npm run build:api</code> .	<code>tests/server.test.ts</code> over real HTTP: health, JSON errors, origin refusal, session requirement, body caps, no server banner or stack trace.
Web tier without credentials New	The in-process API route is deleted. The proxy forwards <code>/api</code> to <code>API_URL</code> ; the web tier's configuration is its origin and that address, and it refuses to start without them.	<code>tests/boundary.test.ts</code> : nothing under <code>app/</code> , <code>components/</code> , <code>proxy.ts</code> or <code>instrumentation.ts</code> imports a server module; only <code>server/handler.ts</code> imports the route table.
Client address New	The proxy deletes any <code>x-client-address</code> a client sent and sets it from the address the edge appended; the API trusts that header alone.	<code>tests/proxy.test.ts</code> : forwarded address passed, forged one dropped, query strings preserved, host:port accepted.

Change	What it is	Enforced by
Prisma data layer New	<code>prisma/schema.prisma</code> introspected from the database, including the partial unique index that stops an agent holding two live customers. The generated client serves records in <code>lib/data/</code> : sessions, staff, the activity log, notices, lookups, deliveries, services, queue membership. Ids that are bigints travel as strings; dates as ISO strings.	<code>tests/prisma-schema.test.ts</code> re-introspects and fails on any difference; <code>tests/data.test.ts</code> covers each repository.
Validated raw results New	Every function call and view read is parsed against a schema in <code>lib/data/rows.ts</code> ; a missing or retyped column fails with the query and row named.	<code>tests/data.test.ts</code> : a row from the view and the same row as JSON both pass; a broken row names the query.
One pool, pinned to UTC New	The Prisma client runs over the same <code>pg</code> pool as the raw queries through the driver adapter, and every connection sets <code>timezone=UTC</code> .	The regression test described in section 4.
Deployment Operations	Three images in the Dockerfile, three services in Compose (only the web tier publishes a port), three services in the Render Blueprint with the secrets on the API service and the migrations in its pre-deploy step.	Documented in <code>docs/render.md</code> , including the one-time Blueprint sync an existing deployment needs.

4. The defect the new tests found

This is the part of the round most worth a reader's attention, because it is the argument for the data-layer tests in one paragraph. When the Prisma client was first wired in, a new test inserted a session that had expired one second earlier and asked the client for it. The client returned it. Investigation showed that on a database whose session time zone is Asia/Dubai, as the company server's is, the client read every `timestampz` four hours late and sent every Date parameter four hours early. Sessions would have stayed valid four hours past expiry, the two-minute lookup reuse window would have been wrong, and every time in the activity log would have shown four hours late. The raw `pg` path was unaffected, which is why nothing earlier had caught it.

The fix is the correct one rather than a patch: every pooled connection now sets its time zone to UTC, which is what the Prisma client assumes and what the raw queries never depended on, because every calendar-day rule in the SQL names Asia/Dubai explicitly. A probe run through the same client before and after shows the read-back timestamp moving from four hours off to identical, and the expired session moving from found to not found. A regression test asserts the session time zone is UTC and that a timestamp read through the client equals the same timestamp read raw.

5. Kept by decision, and why

Decision	What the reviewer wanted	Why it stands
The queue rules stay in PostgreSQL functions	Business logic in TypeScript, reached through Prisma	Issuing a ticket numbers it, inserts it, routes it and raises the change notification as one indivisible step, and calling a customer must guarantee no two agents hold the same ticket. Those guarantees come from one advisory lock and a partial unique index inside the database. Rebuilt in application code they would need interactive transactions with row locks in every path that touches the queue, five to ten round trips per action instead of one, and the lock held several times longer, which is the opposite of what \$07 asked for. Three to four weeks and a downgrade in both safety and throughput. The rules are now reached through typed wrappers with validated results, which is the part of the complaint that was about code quality rather than location.
Migrations stay as checksummed SQL files	Prisma Migrate	Prisma's schema language cannot express the functions, triggers or the partial indexes the engine depends on, so Prisma Migrate would describe less than the SQL does and generate migrations that omit the engine. Prisma describes the result of the migrations instead, and a test proves the description matches. Nothing the reviewer complained about in \$04 (rewritten rules, no checksums, untested parser) depends on which tool applies the files; all three are closed.
Web and API in one repository	Independent deploy schedules	They are separate services with separate build commands and separate images; either can be redeployed alone. A second repository would add a shared-types package and a release coordination problem for one developer and one product. The extraction into another repository is mechanical if a second team arrives.
Three stylesheets, app shell at about 1,000 lines	One styling system, smaller components	Unchanged this round and stated as such. The provably dead blocks were removed this morning; the remaining overlaps change what customers see and are a design decision.

6. Proofs

Claim	Evidence	How to repeat
Nothing regressed	203 tests pass across 19 files in two runtimes; type check and lint clean; three builds succeed; coverage thresholds hold (41.9% lines over the server, data layer, proxy and screens).	<code>npm run typecheck && npm run lint && npm run test:coverage && npm run build && npm run build:api && npm run build:worker</code>
The three services work together	The 30-step smoke test now runs through the web tier to the API service and the scheduler: all 30 steps passed (health and alerts, sign-in, forced password change, presence, Salesforce sandbox probe, QR invite, guest session and lookup, idempotent issue, public status, call, TV board, stale version refused, start, offline-mid-service refused, close, history, receipt, reports and CSV, audit, walk-in, paging validation, 404s, headers, sign-out), and a live change was delivered over the event stream through the proxy, which proves the forwarding path streams	Smoke script against the built artefacts, three processes
The web tier cannot reach the database	No file in the web tier imports a server module, by test; the web configuration schema has no database field; the web image installs no database driver into a code path that runs.	<code>tests/boundary.test.ts</code> , <code>tests/config.test.ts</code>
The API behaves correctly on the wire	Health with request id and no caching; 404 as JSON; 403 <code>BAD_ORIGIN</code> before the body is read; 401 <code>NOT_SIGNED_IN</code> on every staff route; 413 and 415 on oversized and non-JSON bodies; no server banner or stack trace.	<code>tests/server.test.ts</code>
Both data descriptions match the database	Re-introspection equals the committed Prisma schema; the SQL snapshot equals the migrated database; every model is a table and every table a model; the partial unique index is described as partial.	<code>tests/prisma-schema.test.ts</code> , <code>tests/schema.test.ts</code>
The time-zone defect is closed	Probe before: client read 15:42Z for a value the database held as 11:42Z and returned an expired session. Probe after: identical timestamps, expired session not found.	<code>tests/data.test.ts</code> "reads timestamps as the database holds them"
Pages hydrate through the web tier	Served through the proxy: 15 script tags, 15 carrying the request nonce, none without; a headless browser rendered the login form with zero policy violations; the API answered health and a JSON 404 through the same path.	Built web tier on one port forwarding to the built API on another; Microsoft Edge headless with console logging
The forwarded client address drives the per-address limit	Twenty-one sign-in attempts with a different username each, from one edge address: twenty refused as wrong, the twenty-first limited by address. A client that set <code>x-client-address</code> itself stayed in the same budget; a different edge address had its own.	Proof script against the built web tier and API with <code>TRUSTED_CLIENT_IP_HEADER=x-client-address</code>
Throughput is unchanged by the split	200 simultaneous check-ins in 1.6 s, the routing sweep at 186 ms with 200 waiting and 20 agents online, 600 parallel agent actions at a median of 28 ms, on a quiet machine; the same order as this morning's 1.3 s, 96 to 214 ms and 23 ms.	<code>node --env-file=.env.test scripts/load-check.mjs 20 200 20</code>
The browser policy survives the split	Pages still carry a per-request nonce with no inline allowance, and API responses carry no page policy.	<code>tests/proxy.test.ts</code>

7. Deploying it

Render. The Blueprint now declares three services. An existing deployment must sync the Blueprint once after this commit reaches `main`: Render creates the private API service, asks for the secrets once (they move from the web service to the API), and wires the API's private address into the web tier. Until that sync, the new web tier refuses to start because it has no `API_URL`, and Render keeps the previous version serving, so the site stays on the old code rather than failing; the boot validation is doing exactly what it is for. The steps are in `docs/render.md`.

Your own server. `docker compose up -d` builds the three images and starts them on one private network; only the web tier publishes a port, on the loopback address, for a TLS reverse proxy to front. The API never needs a public address.

Any Node host. Three processes from three artefacts: `node dist-api/api.mjs`, `node dist-node/standalone/server.js` with `API_URL` pointing at the first, and `node dist-worker/worker.mjs`.

8. What remains

Item	Type	Owner
Routing policy: cross-service fairness, new-member position, cap on uncalled assignments, owner-first threshold, priority flag; then a settings page for the agreed knobs	Decision	CRM and Collection managers, Product
Retention periods; single sign-on or multi-factor; HR-linked offboarding	Decision	Product, IT
Production PostgreSQL server with backups and a rehearsed restore; production Salesforce org and integration user; production secrets; uptime monitor on the alert endpoint; device acceptance	Operations	IT, Salesforce team, CRM team
If production serves several branches: a branch column, one lock and numbering per branch, change events scoped per branch	Engineering, three to five days, before a multi-branch launch	Developer
Component tests for the remaining screens; one stylesheet; metrics and tracing	Engineering, not blocking	Developer

9. How the next reviewer verifies this

```
git clone https://github.com/pruthviiii/Samana-QMS.git && cd Samana-QMS && npm ci
# any PostgreSQL 14+: create samana_qms_test; put its URL in .env.test with
# APP_ORIGIN=http://qms.test, SESSION_COOKIE_SECURE=false, API_URL=http://127.0.0.1:3001
node --env-file=.env.test scripts/migrate.mjs
npm run typecheck && npm run lint && npm run test:coverage
npm run build && npm run build:api && npm run build:worker
# boundary: grep -rn "lib/db\|lib/prisma\|lib/api" app components proxy.ts instrumentation.ts    (nothing)
# drift:     npx prisma db pull --print --url "$DATABASE_URL" | diff - prisma/schema.prisma    (empty)
# wire:      node --env-file=.env.test dist-api/api.mjs & curl -i localhost:3001/api/health
```

10. Change log

Commit	Date	Scope
2341d2a	22 Sep	Architecture review change set described; report stamped
9295489	22 Sep	API extracted into its own private service on a Node HTTP host; web tier reduced to screens and a forwarding proxy with no credentials; client address carried as x-client-address; Prisma 7 schema introspected and pinned by a drift test; typed data layer in lib/data with validated raw results; one UTC-pinned pool shared by the Prisma client and raw queries; time-zone defect found and fixed with a regression test; server, data-layer, configuration and proxy tests; three build artefacts, Compose services and Render services; documentation and runbooks

Companion documents: [docs/review-response-2026-09-22.html](#) (response to the first review), [docs/architecture-review-2026-09-22.html](#) (the morning's review), [docs/review-checklist-2026-09-21.md](#) sections I and J, [docs/requirements.md](#) decisions, [docs/render.md](#), [.env.example](#), and the design document "Samana QMS – Demo Flow and Technical Design".